Midterm Part 1: The Sensor

Start Assignment

- Due Tuesday by 5pm
- Points 40
- Submitting a file upload
- Available after Oct 12 at 2pm

Midterm Part 1: The Sensor

> Upload your code before you demonstrate.
>
> Your two files go to **this assignment**, and they must be showing as submitted in Canvas **before** the TA starts your week 1 demonstration.
>
> A demonstration with no code submitted scores **zero** for Part 1, whatever the bench did.

Bring up the LSM303 and prove the numbers are real. **40 points, demonstrated at the end of week 1.**

> Part 2 measures the bus that this traffic runs on, so getting the sensor talking this week is what makes next week possible. It is also why the deadline is where it is.

1a. Build a known-answer test (10 points)

In Lab 5 you read the ADT7420's identification register before its temperature, because a fixed constant is either exactly right or wrong and a temperature only ever looks plausible.

The LSM303DLHC accelerometer has no identification register. There is no fixed constant to read back from it. So construct the equivalent test yourself, in two steps.

Step 1: read a value you did not choose

- Find a register whose **power-on default is documented in the datasheet**, and pick one whose default is **not zero**.
- Read it. Report the register address, the documented default, and what you actually got.

> The non-zero part matters. A read path that returns 0x00 for everything is a common failure mode, and it would sail through a test against a register whose default happens to be zero.

Step 2: write a value you did choose, and read it back

- Write a different, recognisable value to that same register.
- Read it back. It must return exactly what you wrote.
- Report both, and show the transaction working.

Why two steps, and why this is a better test than an ID register would have been

Step 1 proves your **read** path works against a value you had no hand in choosing. Step 2 proves your **write** path works and that you are addressing the register you think you are.

This part will not produce any data until you configure it, so the write path is not optional here — and an identification register, had one existed, would have tested only half of what you need. The absence of a WHO_AM_I is an inconvenience that happens to force a more complete test.

> The magnetometer, unlike the accelerometer, does have identification registers. Reading them is a legitimate additional check and worth doing — but note carefully what it proves: that the MAGNETOMETER's address and read path work. It says nothing about the accelerometer, which is a different device at a different address.

1b. Wake it up and read three axes (5 points)

- Write whatever control register is needed to take the accelerometer out of its power-up state.
- Read all six output bytes in a single burst and assemble three signed values.
- Convert to g using the full-scale setting and scale factor from the datasheet. State both in Report 1.
- Print ax, ay, az continuously and show that they respond sensibly when you tilt the board.

> If all three axes return identical values, stop and re-read the section of the datasheet about multi-byte reads. That symptom has exactly one cause.

1c. The gravity check, and breaking it deliberately (20 points)

This is the part that carries the marks.

Place the board at rest in **six orientations** — each of its faces and edges, such that gravity points along a different axis each time, in both directions. For each orientation, average enough readings to get a stable value and record:

Orientation	ax (g)	ay (g)	az (g)	|a| (g)

1 … 6

What is required

- In each orientation, **one axis should read close to ±1 g and the other two close to zero**. Work out from your own data which physical direction each axis corresponds to; do not assume.
- The magnitude |a| must come out to **1 g in all six orientations**.
- **Report the spread**: the largest |a| minus the smallest, across the six. That number is your result. A tight spread is the evidence that the read is correct.

The spread matters more than the average

A wrong sign, a swapped byte or a missing auto-increment can still produce an average near 1 g if you are unlucky. What none of those faults can do is hold |a| constant while the board rotates through six orientations. That is why six, and not one.

Acceptance

The spread in |a| across the six orientations must be **under 0.15 g**. That is a loose bound on purpose: it catches every gross fault while leaving room for the sensor's own uncalibrated offset, which Report 1 asks you to account for separately.

Then break it on purpose

Once the six orientations pass, take your working code and introduce **one specific fault**: clear the bit that makes the register pointer advance during a burst read. Change nothing else.

- Repeat **two** of your six orientations and record ax, ay, az and |a|.
- Describe the signature in one sentence, and show that it matches what the arithmetic predicts.
- Put the bit back.

> Anyone can avoid a bug they were warned about. Reproducing it deliberately, predicting its signature, and recognising it in your own data is a different thing, and it is the thing worth having when the next silent failure is one nobody warned you about.

1d. Sustain 100 samples per second (5 points)

A hundred three-axis samples per second is the rate a motion-tracking application needs to feel responsive rather than laggy. This is not a measurement exercise — it is a requirement, and it is here to force a design decision.

- Demonstrate **at least 100 samples per second, sustained for at least 30 seconds**, with all three axes read each time.
- Report your SCL frequency, the bytes per transaction, and where the remaining time goes.
- Show that your samples are **independent**: that each one is a new conversion rather than a re-read of a value the sensor has not updated. Say how you established that.

Both halves of this bite

Reading the six output bytes as six separately addressed transactions will not reach 100/s at any SCL frequency you are likely to be running. Reading them as one burst will, comfortably. The requirement exists to force that design decision rather than leave it to chance.

And polling faster than the sensor produces data gives you numbers without giving you information — exactly the trap from Lecture 7, in a new place. The output data rate is a setting you control. Choose it, and then prove you chose it correctly.

The magnetometer (no marks, but worth doing)

Configure the magnetometer and read its three axes as well. It has its own address, its own control register, its own conventions for byte and axis order, and — unlike the accelerometer — a set of identification registers you can check against the datasheet.

It is not graded, for a reason worth knowing: the Earth's field is weak, and nearby steel, the board's own currents and anything with a motor in it will swamp it. It is a much poorer referee than gravity. Get it talking, look at the numbers, and note in Report 1 what you observed — particularly whether the field magnitude stays constant as you rotate the board, and if not, why not.

Demo & Submission

Upload **two separate files** to this assignment, with exactly these names. Not zipped, not pasted into a document.

Filename	Contents

`midterm_p1_i2c.v`	your I²C state machine, with whatever you changed to reach the new device
`midterm_p1_python.py`	the host code: the known-answer test, the three-axis read, the six orientations and the 100/s run

Your written work goes to the separate **Midterm – Report 1** assignment, due at the same moment. Code does not go there.

> Upload your code before you demonstrate.
>
> The TA checks Canvas before starting and will not begin until your files are showing as submitted.
>
> A demonstration with no code submitted scores **zero** for this part, whatever the bench did. If something is missing or misnamed when the TA looks, fix it there and then — caught at the bench it costs thirty seconds.

Due: First 60 minutes of the session at the end of week 1, the same deadline as Report 1. Late submission costs 10% per hour.

What the TA Checks, at the end of week 1

#	The TA does this	This must happen
0	**Opens this assignment in Canvas before starting**	**both code files are submitted and correctly named.**
1	Watches your known-answer test run	the documented power-on default reads back, and a value you wrote reads back unchanged
2	Asks you to tilt the board	live ax, ay, az respond correctly and sensibly
3	Looks at your six-orientation table	all six present, spread in |a| stated and under 0.15 g
4	Looks at your broken-on-purpose run	two orientations repeated with the auto-increment bit cleared, and the signature described
5	Watches your sustained rate measurement	at least 100 samples/s held for 30 seconds, all three axes each time

6	Asks how you know the samples are independent	a stated method, not an assumption

Check 6 is the one people skip. Polling faster than the sensor produces data gives you numbers without information, and nothing in the output says so.